

PROJETO 04 DE 10

# Deploy em Kubernetes Local (kind / minikube)

Avançado

Kubernetes · Docker · kind · Helm

Guia de estudo DevOps & Cloud · Junho de 2026

## Projeto 4 — Deploy em Kubernetes Local (kind / minikube)

Avançado

Containerizar uma aplicação e fazer o deploy num cluster Kubernetes rodando na sua própria máquina — sem gastar com nuvem. Kubernetes é a skill nº1 em vagas DevOps.

Kubernetes

Docker

kind

Helm

### O que o projeto faz

Você cria um cluster Kubernetes local (com kind ou minikube), constrói a imagem Docker da sua app e descreve, em arquivos YAML, como o Kubernetes deve rodá-la: quantas réplicas, como expor na rede, como reiniciar se cair. É o mesmo modelo usado em produção, só que local.



### O que é usado (stack)

- **Kubernetes (kubectl)** — orquestrador de containers.
- **kind ou minikube** — cluster K8s local.
- **Docker** — construir a imagem da app.
- **Helm** (opcional) — empacotar os manifests.

### Estrutura do projeto

```
k8s-deploy/
├── app/
│   └── Dockerfile      # imagem da aplicação
└── k8s/
    ├── deployment.yaml # como rodar a app (réplicas, imagem)
    ├── service.yaml    # como expor na rede
    └── ingress.yaml     # roteamento por URL (opcional)
```

### Código comentado

#### k8s/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment      # tipo de objeto: gerencia réplicas
metadata:
  name: minha-app
spec:
  replicas: 2          # mantém 2 cópias (pods) sempre
  selector:
    matchLabels: { app: minha-app } # quais pods pertencem a este Deployment
  template:            # "molde" de cada pod
    metadata:
      labels: { app: minha-app }
    spec:
      containers:
        - name: web
          image: minha-app:1.0    # a imagem construída
          ports:
            - containerPort: 8080
          readinessProbe:          # K8s só manda tráfego quando estiver pronto
            httpGet: { path: /health, port: 8080 }
```

O que cada parte faz: o **Deployment** garante que sempre existam **replicas: 2** pods no ar — se um cair, ele recria. O **selector / labels** é como o Deployment "acha" seus pods (precisam casar). O **readinessProbe** faz o Kubernetes só enviar tráfego quando a app responder em **/health** — evita servir requisição numa app que ainda está subindo.

## k8s/service.yaml

```
apiVersion: v1
kind: Service
metadata: { name: minha-app-svc }
spec:
  selector: { app: minha-app } # encaminha p/ os pods com este label
  ports:
    - port: 80
      targetPort: 8080 # porta do container
  type: ClusterIP # acessível dentro do cluster
```

O **Service** dá um "endereço fixo" para um conjunto de pods que podem nascer e morrer. Ele encontra os pods pelo mesmo **selector** de labels.

## Comandos

```
kind create cluster # cria o cluster local
docker build -t minha-app:1.0 ./app
kind load docker-image minha-app:1.0 # leva a imagem p/ dentro do kind
kubectl apply -f k8s/ # cria Deployment + Service
kubectl get pods # vê os pods
kubectl logs <pod> # vê os logs
kubectl port-forward svc/minha-app-svc 8080:80 # acessa local
```

## Explicação linha a linha — deployment.yaml

|    |                                        |                                                            |
|----|----------------------------------------|------------------------------------------------------------|
| 1  | apiVersion: apps/v1                    | Versão da API do Kubernetes para este tipo de objeto.      |
| 2  | kind: Deployment                       | O tipo de objeto: um Deployment gerencia réplicas de pods. |
| 3  | metadata:                              | Início dos metadados (identificação).                      |
| 4  | name: minha-app                        | Nome do Deployment.                                        |
| 5  | spec:                                  | Especificação do comportamento desejado.                   |
| 6  | replicas: 2                            | Mantém sempre 2 cópias (pods) rodando.                     |
| 7  | selector:                              | Como o Deployment identifica os pods que são dele.         |
| 8  | matchLabels: { app: minha-app }        | Procura pods com o label app=minha-app.                    |
| 9  | template:                              | O "molde" usado para criar cada pod.                       |
| 10 | metadata:                              | Metadados do pod.                                          |
| 11 | labels: { app: minha-app }             | O label PRECISA casar com o selector acima.                |
| 12 | spec:                                  | Especificação do conteúdo do pod.                          |
| 13 | containers:                            | Lista de containers do pod.                                |
| 14 | - name: web                            | Nome do container.                                         |
| 15 | image: minha-app:1.0                   | A imagem Docker que será executada.                        |
| 16 | ports: [containerPort: 8080]           | Porta que o container expõe.                               |
| 17 | readinessProbe:                        | Checagem de prontidão: K8s só envia tráfego quando passar. |
| 18 | httpGet: { path: /health, port: 8080 } | Faz GET em /health para saber se está pronto.              |

### Erros comuns na criação

- **ImagePullBackOff / ErrImagePull:** o cluster não acha a imagem. Em kind, use `kind load docker-image`; defina `imagePullPolicy: IfNotPresent`.
- **CrashLoopBackOff:** a app sobe e cai. Veja `kubectl logs` — geralmente erro de config/porta.
- **Pod Pending:** falta recurso ou o selector não casa. Cheque labels e `kubectl describe pod`.
- **Não consigo acessar a app:** Service ClusterIP só é interno. Use `port-forward` ou um Ingress.
- **selector não bate com labels:** erro silencioso — o Deployment fica sem pods. Confira se os labels são idênticos.

#### Para aprender mais

- Entenda os objetos: **Pod**, **Deployment**, **Service**, **Ingress**, **ConfigMap**, **Secret**.
- Aprenda **Helm** para reaproveitar e parametrizar manifests.
- Esse projeto é a base do Projeto 10 (mesma coisa, mas no EKS da AWS).